

Lesson 3 — What Each Notebook Shows

Notebook guide, lesson 3 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

16 October 2026 · three notebooks

Contents

Notebook 01 — 01_linear_regression_from_scratch	1
Linear regression, built from nothing	1
Notebook 02 — 02_polynomial_and_overfitting	2
Curves, and the price of flexibility	2
Notebook 03 — 03_ridge_and_lasso	3
Ridge and Lasso: paying for large coefficients	3

A short guide to this lesson's notebooks. For each one: what it is for, the concepts in the order they appear, the numbers worth noticing, and **the two-minute version**, a summary to read before running it or to come back to afterwards. Every number here comes from the committed output of the notebook it belongs to.

Notebook 01 — 01_linear_regression_from_scratch

Linear regression, built from nothing

What it is for. By the end you have written the model, the cost function, the exact solution and the iterative one — and checked all four against scikit-learn. It is the lesson's only “build it yourself” notebook, and the point is that nothing in a fitted linear model is mysterious.

The data. 600 generated houses, six measurements and a price, with the generating coefficients available **for checking only**: +2,400 €/m², +9,000 per bedroom, +14,000 per bathroom, −1,800 per year of age, −6,500 per km from the centre, +12,000 for a garage, intercept 45,000.

The concepts, in order.

1. **What a linear model claims**, read as a sentence: every extra square metre adds 2,400 euros. A fixed price per unit, whatever the size.
2. **The cost surface.** Three guesses at (w, b) with costs of 12.7 billion, 1.1 billion and 2.2 billion — fitting means finding the lowest point of that surface.

3. **The exact answer.** The normal equation on area alone: $b = -51,025$, $w = 2,785$ €/m², and agreement with scikit-learn to 1.8×10^{-12} .
4. **A worked check worth making aloud.** The fitted slope is 2,785 against a true 2,400 — **16% too high**, because area alone has to absorb the effect of everything correlated with it. Not an error; an omitted-variable effect.
5. **The iterative answer.** Gradient descent after its run: $w = 2,368$, $b = -1,351$. The slope has essentially converged and **the intercept is still crawling** — lesson 2's conditioning argument arriving exactly as predicted.
6. **All six features**, where the omitted-variable bias disappears:

	true	estimated	error %
area_sqm	2400	2421	+0.9
bedrooms	9000	7503	-16.6
age_years	-1800	-1791	-0.5
distance_km	-6500	-6455	-0.7

bedrooms stays off because it correlates 0.77 with area — the two cannot be separated cleanly.

7. **Does the model hold?** Residual standard deviation **20,340 €** against the **18,000 €** of noise actually added. The excess is the model's own estimation error, and it shrinks with more data.
8. **Against what? The regression baseline.** `DummyRegressor(strategy="mean")` scores a root mean squared error (RMSE) of **96,440 €** against the model's **20,341 €**, and the same comparison as one ratio is $R^2 = 0.956$ — computed both by hand from the definition and with `r2_score`. Lesson 1's notebook 1 defined R^2 ; this is where the course first needs it, because it is the baseline comparison the classification lessons make automatically.

The two-minute version. *We build linear regression from nothing on generated house prices, so the true coefficients are known. Fit area alone and the normal equation gives 2,785 euros per square metre where the truth is 2,400 — 16% high, because area is standing in for everything correlated with it. Our implementation and scikit-learn agree to twelve decimal places. Gradient descent reaches the same slope but its intercept is still crawling, which is the scaling argument from last week showing up in the iterations. With all six features the estimates land within 1% of the truth, except bedrooms, which correlates 0.77 with area. And the residual spread, 20,340 euros, sits just above the 18,000 of noise we added — that gap is what the model does not know. The last section asks the question lesson 1 taught them to ask: against what? Always predicting the mean costs 96,440 euros against our 20,341, and R-squared is exactly that comparison written as one number, 0.956.*

Notebook 02 — 02_polynomial_and_overfitting

Curves, and the price of flexibility

What it is for. Notebook 1 fitted a line to data that really was linear. Real relationships bend, a linear model can be made to bend, and this notebook measures what that costs.

The data. Energy consumption against outdoor temperature — heating in the cold, cooling in the heat — with 21 training points and 9 test, generated from $240 + 1.15(t - 18)^2$ plus noise of 22 kWh.

The concepts, in order.

1. **Linear in the coefficients, not in the inputs.** Hand the model t^2 as an extra column and it is still least squares, still the same normal equation.
2. **Degree 1 against degree 2.** Test RMSE **212.1** against **17.4** — and the degree-1 residuals show a clear **U shape**, which is the diagnostic: structure the model did not take. Notebook 1's residuals had no such pattern.
3. **Too much flexibility**, the table that carries the lesson:

degree	train RMSE	test RMSE
1	113.3	212.1
2	17.6	17.4
3	17.5	16.9
6	17.0	23.3
9	16.2	590.1
12	13.6	24,655.7

Training error falls at every single step while test error turns around at degree 3. Training error cannot choose a model.

4. **Why the wiggles appear.** Not because 12 is a large number: because **thirteen coefficients on 21 points** can be arranged to pass near all of them, and that arrangement must swing violently between them. Largest coefficient at degree 2: **411**. At degree 12: **3,097,038,010**.

The two-minute version. *A relationship that bends, and 21 points to learn it from. A straight line leaves a test error of 212; degree 2 brings it to 17, and the diagnostic is in the residuals — the line leaves a U-shaped pattern, the quadratic leaves none. Then we push the degree up and read the two columns against each other: training error falls at every step, from 113 to 14, while test error turns around at degree 3 and reaches 24,656 by degree 12. The mechanism is in the coefficients: 411 at degree 2, three billion at degree 12, working in opposition. That number is the symptom that notebook 3 treats.*

Notebook 03 — 03_ridge_and_lasso

Ridge and Lasso: paying for large coefficients

What it is for. Notebook 2 ended with a diagnosis — overfitting shows up as enormous opposing coefficients. If that is the symptom, penalising size is the treatment.

The concepts, in order.

1. **The two penalties.** Ridge charges $\sum_j w_j^2$, Lasso charges $\sum_j |w_j|$, and λ (scikit-learn's **alpha**) sets the exchange rate.

2. **Ridge against the degree-12 disaster**, and the table to read from the right:

model	train RMSE	test RMSE	largest $ w $
no penalty	13.6	24,655.7	3,097,038,010
ridge, $\lambda = 0.01$	18.0	22.8	365
ridge, $\lambda = 1$	44.5	105.2	157
ridge, $\lambda = 100$	96.0	227.6	6.3

A penalty of 0.01 — barely a touch — takes the largest coefficient from three billion to 365 and the test error to the noise floor. This is the lesson’s number to remember.

3. **The regularisation path.** Every coefficient tracked as λ sweeps. Ridge coefficients approach zero smoothly and never arrive; **Lasso coefficients reach zero at a finite λ and stop** — which is what makes it a selector.
4. **The order of elimination is not arbitrary:** 6 features kept at $\lambda = 1,000$, then 5, then 3, then only `area_sqm` at $\lambda = 40,000$. `garage` goes first, being the smallest effect once everything is on a common scale.
5. **The case Ridge was invented for.** Two columns that are the same measurement in different units, correlation **0.999999**. Least squares gives them +24,891 and −2,087, summing to 2,424 against a true 2,400: **right about the price, wrong about why, and violently unstable**. Across four splits the `area_sqm` coefficient goes $9,261 \rightarrow 14,535 \rightarrow 45,307 \rightarrow 39,061$. Ridge gives the two columns nearly identical coefficients on every split, around 38,000–41,000 each.

The two-minute version. *Overfitting looked like a three-billion coefficient, so we put a price on size. Ridge charges the squares, Lasso the absolute values. On the degree-12 disaster a penalty of one hundredth takes the largest coefficient from three billion to 365 and the test error from 24,656 to 23, which is the noise floor. The regularisation path shows the difference between them: Ridge coefficients shrink towards zero forever, Lasso coefficients reach zero and stop, which is why Lasso selects features — and the order it drops them in is the order of their effect size. The last section is the case Ridge was designed for: two columns of the same measurement in different units, where least squares invents two huge opposing coefficients that swing by a factor of five between splits, and Ridge splits the effect evenly and stays put.*